

Verifier-Assisted versus LLM-Only Pre-deployment Network-Change Validation: A Preregistered Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We measured whether integrating a deterministic pre-deployment verifier with a bounded automated repair budget increases the fraction of intent-driven network-change proposals that pass semantic validation compared with accepting a single LLM candidate. In a preregistered paired design (study id `cnsmp2026-llm-verifier-predeployment-001`) we executed 300 paired intents (100 per difficulty stratum: low/medium/high) using `gpt-5-mini` and `deterministic_netops_validator_v5`. Each paired intent produced one shared_initial generation that was evaluated under two conditions: baseline (accept single shot) and guarded (deterministic validation and ≤ 1 automated repair). Outcomes: baseline 299/300 unflagged passes (0.9966667), guarded 300/300 (1.0); contingency $n_{11}=299, n_{10}=1, n_{01}=0, n_{00}=0$; absolute paired difference = 0.0033333; exact McNemar two-sided $p = 1.0$; 95% paired bootstrap percentile CI = [0.00,0.01] (bootstrap_seed = 1729, B = 10000). Under the supplied synthetic suite and repair budget the single-shot generator was near ceiling, limiting observable deterministic rescue. We archive preregistration, execution, and analysis artifacts and interpret results with respect to estimand conditioning, validator coverage, repair budget, and operational deployment policies.

I. INTRODUCTION

Generative large language models (LLMs) are being applied to network operations (NetOps) tasks such as intent→configuration translation, configuration synthesis, and automated repair. For pre-deployment network changes, semantic correctness properties—sequencing constraints, reachability, and policy preservation—are operationally critical and cannot be assured by superficial syntactic checks alone. A common engineering pattern is to pair generative models with deterministic validators and bounded automated repair (generate→validate→repair) to reduce operational risk. However, the measurable benefit of such guarded pipelines depends on the experimental estimand (what is held fixed), the generator single-shot quality, the validator’s expressiveness, and the permitted repair budget.

We preregistered a paired experiment (study id `cnsmp2026-llm-verifier-predeployment-001`) to quantify the deterministic rescue achievable when the same single LLM candidate is evaluated under two treatments. For each intent the pipeline produced exactly one shared_initial candidate using the declared generator (`gpt-5-mini`) and compared: (a) baseline — accept the single candidate without repair, and (b) guarded — run a deterministic validator (`deterministic_netops_validator_v5`) and allow at most one automated repair which itself is re-validated. The primary research

question: does adding a deterministic verifier plus at most one automated repair increase the probability that a single LLM candidate passes semantic pre-deployment validation compared with accepting that same candidate without repair?

Contributions of this artifact-grounded study: (1) a preregistered paired evaluation isolating deterministic rescue under a shared_initial estimand that conditions on a single LLM draw per intent; (2) a complete execution and analysis bundle including validator traces and the single automated repair transcript archived with the run; and (3) an evidence-grounded interpretation of estimator conditioning, ceiling effects, and operational implications for NetOps pipelines.

II. RELATED WORK

Prior work on LLMs for network management has pursued two complementary directions that frame our study. First, multiple recent surveys and systems reports document active efforts to apply LLMs to NetOps tasks including intent translation, template synthesis, protocol test generation, and zero-touch workflows [<https://openalex.org/W4402742293>; <https://openalex.org/W4411015066>; <https://openalex.org/W4415275309>]. These surveys synthesize a range of prototypes and experimental systems showing that LLMs can produce syntactically plausible device configurations and candidate workflows, but that practical deployment depends critically on downstream validation and environment modeling. For example, PreConfig (a pretrained model for automating configurations) and other synthesis systems demonstrate automation potential while also highlighting dataset and device-scope differences that limit straightforward generalization [<https://openalex.org/W4392875514>; <https://openalex.org/W4411015066>].

Second, a line of systems research specifically proposes and evaluates tool-augmented LLM architectures that interpose external checkers, planners, or verifiers. Planner-centric and ReAct-style frameworks explicitly enable LLMs to call validators or external tools to check semantics and coordinate multi-step changes; these architectures aim to reduce hallucination and logical errors by shifting some checks outside the model [doi:10.1609/aaai.v40i40.40676; <https://openalex.org/W4389518608>]. Empirical system reports in the NetOps space have implemented generate→validate→repair patterns and reported

task-dependent improvements in correctness when validators capture operational invariants: MeshAgent and related systems show that tool augmentation can improve reliability for certain configuration tasks, while program-repair-oriented work demonstrates that combining symbolic fault localization with generative repair has practical value on constrained benchmarks [<https://openalex.org/W4416927413>; <https://openalex.org/W4404240614>].

Work on error detection and mitigation complements these architecture proposals. Detection methods grounded in model uncertainty or semantic measures (for example, entropy-based detectors) and deterministic safety-net designs have been proposed to catch hallucination or logical inconsistencies before deployment; empirical evaluations indicate these detectors can reduce false assurances but are sensitive to detector design and the class of errors targeted [<https://openalex.org/W4399803256>; doi:10.2139/ssrn.6538460]. Similarly, domain-targeted testing work has shown that LLM agents can generate protocol test cases or device-specific actions, but that correctness requires encoding topological and sequencing invariants into validators or test harnesses to catch semantic failures [<https://openalex.org/W4415275309>].

Two methodological gaps in the literature motivate our paired shared_initial design. Many prior evaluations conflate gains from resampling (drawing multiple independent LLM candidates) with gains that arise solely from deterministic validation and repair; permissive generation policies permit the former and thus measure a different operational tradeoff than a single-call budget. Second, while some works evaluate generate→validate→repair pipelines on task-specific suites, fewer public benchmarks and systematic comparisons isolate semantic correctness (reachability, sequencing, forbidden-template preservation) as a first-class evaluation target across difficulty strata [<https://openalex.org/W4411015066>; <https://openalex.org/W4392875514>]. Our study explicitly conditions on a single shared_initial candidate per intent and uses a deterministic validator with an archived, bounded repair operator to provide an operationally meaningful bound on deterministic rescue that complements existing multi-call or planner-guided evaluations.

III. METHODOLOGY

Preregistration and estimand. The study preregistration (cnsmp2026-llm-verifier-predeployment-001; preregistration SHA-256 = 425cb1652354120cf48f686b964723d64c76d0a8abe53cd5b5ad0ea1af71fc7) fixed the primary estimand as the absolute paired difference in the unflagged verifier-pass proportion (guarded minus baseline) computed on 300 paired cases after applying inclusion/exclusion rules. The confirmatory analysis executor paired_binary_analysis_v1 and all inclusion/exclusion rules were frozen prior to the main run. An optional pilot (n = 60) was described in the preregistration for discordance estimation; that pilot was not executed and no post-preregistration power recalculation was performed

(this deviation is recorded in the archived preregistration and Disclosure Statement).

Task generation and synthetic suite. Tasks were produced deterministically by the registered deterministic_netops_task_generator_v5 into a synthetic_topology_suite stratified into low/medium/high difficulty bins by generator metadata (changed_interface_count, dependency_rule_count, required_command_count). Each task manifest entry encodes the full intent, initial and expected final states, required_sequence constraints, preserve_unspecified_state invariants, and protected_interfaces. Representative difficulty patterns include 'shutdown_configure_restore' (medium) and 'paired_link_atomic_mtu_change' (hard). The synthetic suite enforces validator-checkable invariants used for scoring.

Shared_initial protocol and generator. To implement the shared_initial estimand the pipeline produced exactly one shared_initial generation per pair and reused that identical candidate for both baseline and guarded conditions. The declared generator for shared_initial candidate production was gpt-5-mini (execution/model_configuration.json recorded requested_model = "gpt-5-mini", declared_model_version = "gpt-5-mini"). This isolates deterministic validator/repair rescue from gains achievable by independent resampling, verifier-guided generation, or iterative generation.

Validator, scoring, and repair operator. deterministic_netops_validator_v5 executed the preregistered full_intent_constraint_validation rule: deterministic checks for sequencing invariants (must_be_down_for_fields), required_command_count, preservation of unspecified state, topology reachability invariants encoded in the synthetic suite, and forbidden-template protection for protected_interfaces. Scoring was binary: unflagged == pass. The guarded pipeline permitted at most one automated repair attempt (repair budget <=1). The repair routine was a constrained edit operator limited to localized sequencing or insertion edits; it logged a repair provider trace and was applied only when the validator flagged the shared_initial candidate. Candidates remaining flagged after the permitted repair counted as failures for the primary estimand.

Contamination controls, negative controls, and stopping rules. The preregistration included one intentionally broken negative control per complexity stratum and deterministic invariant checks (pre/post reachability matrices). Per preregistration, pairs with both calls failing due to provider/infrastructure outages would be excluded (none occurred). The preregistered stopping rule would halt if >20% dual-call exclusions occurred; this did not trigger.

Execution instrumentation and audit. The confirmatory run used adapter hosted_netops_gvr_v1 under execution_mode = scientific_confirmatory. Execution manifest values (archived): planned_episode_count = 600, completed_episode_count = 600, complete_pair_count = 300, model_calls_used = 301, maximum_model_calls = 600. Provider call auditing recorded hosted_model_call_event_count = 301, cache_miss_count = 301, stage_counts: shared_initial_generation = 300 and repair = 1. Deterministic reconciliation confirms

episode and provider-call accounting and reports `all_deterministic_consistency_checks_passed = true`. These audit values are archived for third-party verification.

Analysis executor and inferential settings. The preregistered analysis executor `paired_binary_analysis_v1` computed paired contingency counts ($n_{11}, n_{10}, n_{01}, n_{00}$), the absolute paired difference (guarded - baseline), an exact two-sided McNemar test, and a paired bootstrap percentile 95% CI. The bootstrap parameters archived with the analysis are `seed = 1729` and `resamples B = 10000`. Sensitivity analyses specified in the preregistration (failure-as-zero and stratum-wise tests with Benjamini-Hochberg FDR control) were run as exploratory artifacts; the confirmatory results reported here follow the primary analysis plan.

IV. RESULTS

Execution integrity and accounting. The preregistered adapter completed the confirmatory run with `completed_episode_count = 600`, yielding `complete_pair_count = 300` paired observations under the `shared_initial` protocol. Provider/audit records show `hosted_model_call_event_count = 301` (300 `shared_initial` generations plus one repair invocation), `cache_miss_count = 301`, and `stage_counts` indicating `shared_initial_generation = 300` and `repair = 1`. Deterministic reconciliation confirmed consistent episode accounting (`all_deterministic_consistency_checks_passed = true`) and the absence of provider/infrastructure failures or excluded pairs.

Primary confirmatory outcomes. The binary scoring produced `baseline_success_count = 299` and `guarded_success_count = 300` (`analysis/condition_summary.csv`), i.e., `baseline = 299/300` (0.9966666667) and `guarded = 300/300` (1.0). The paired contingency table (`analysis/paired_contingency_table.csv`) is $n_{11} = 299$, $n_{10} = 1$, $n_{01} = 0$, $n_{00} = 0$. The preregistered exact two-sided McNemar test (operating on the discordant counts n_{10} and n_{01}) returned $p = 1.0$. The absolute paired improvement (guarded - baseline) equals 0.0033333333333332993. The paired bootstrap percentile 95% confidence interval computed with `seed = 1729` and `B = 10000` is [0.00, 0.01]. Because discordance is minimal ($m = n_{10} + n_{01} = 1$), the exact test has limited sensitivity and the bootstrap CI bounds the plausible absolute improvement to at most one percentage point in this experimental context.

Per-stratum diagnostics. Per the task manifest stratification, archived per-stratum tallies were: low difficulty `baseline = 100/100` and `guarded = 100/100`; medium `baseline = 100/100` and `guarded = 100/100`; high `baseline = 99/100` and `guarded = 100/100`. Thus all discordant evidence is concentrated in the high stratum (1 rescues), while low and medium strata were at ceiling under both conditions. These counts are reported in the archived `condition_summary.csv` and corroborate that the observed discordance derives from a single hard example under the synthetic generator’s difficulty taxonomy.

Repair diagnostic and episode-level trace. The solitary discordant pair invoked the permitted automated repair. Validator

traces (`deterministic_netops_validator_v5`, `validator_version = 5.0`) attribute the initial failure to a sequencing invariant violation (the preregistered `must_be_down_for_fields` rule): the `shared_initial` candidate omitted an explicit ‘admin down’ step prior to a VLAN/MTU change required by the intent. The constrained repair operator performed a single localized insertion of the missing sequencing command, after which the deterministic validator reported `valid == true` for the corrected candidate. The repair therefore produced a deterministic rescue with one additional model/repair stage call. The repair provider trace and the validator’s before/after traces for that episode are archived; the manuscript conveys the semantic summary rather than reproducing verbatim provider logs in the body.

Contamination, missingness, and negative controls. The preregistered contamination checks produced an empty `contamination_summary.csv` (no contamination flags). The missingness summary reports zero failed or unscorable episodes and zero excluded pairs under preregistered rules (`analysis/missingness_summary.csv`). Negative controls included by the preregistration did not cause unexpected pass behavior in the main run; any negative-control anomalies would have been treated as contamination per the preregistration and are not present in the archived run.

Cost and operational footprint. The guarded pipeline’s deterministic rescue required a single additional repair stage call across 300 pairs (`model_calls_used = 301`), indicating that bounded automated repair under the chosen edit operator imposes minimal extra model-call cost in this run. The empirical `repair_stage_count = 1` (analysis artifact) quantifies the low marginal runtime/model cost for deterministic rescue under the chosen repair budget and edit operator design.

Statistical interpretation and limitations of inference. The confirmatory inferential outcome ($p = 1.0$) primarily reflects the paucity of discordant pairs under the `shared_initial` estimand and chosen synthetic suite. Exact McNemar inference depends only on n_{10} and n_{01} ; with $n_{10} = 1$ and $n_{01} = 0$ the two-sided exact p equals 1.0, indicating no statistically detectable difference under conventional two-sided testing in this sample. The paired bootstrap CI (`seed = 1729`, `B = 10000`) provides a complementary bound for effect size: the absolute paired improvement is plausibly between 0 and 0.01 given observed data and the preregistered estimand. These inferential statements apply strictly to the `shared_initial` conditioning (one LLM draw reused across conditions), the limited repair budget (≤ 1 constrained edit), the synthetic task distribution, and the declared generator (`gpt-5-mini`) and validator (`deterministic_netops_validator_v5`).

Practical diagnostic takeaway. Under operational policies that restrict model calls per change request, the guarded `generate→validate→repair` pattern produced a deterministic rescue in one hard example with negligible additional model-call expenditure in this synthetic suite. However, because baseline single-shot generation was near ceiling (299/300), the marginal observable benefit of bounded deterministic repair was correspondingly small in absolute terms. The archived per-episode validator traces and repair transcript

enable forensic inspection to inform whether richer repair operators, iterative repair budgets, or alternative estimand conditioning (e.g., allowing resampling) would be expected to materially increase rescue rates in different deployment regimes.

V. DISCUSSION

Estimand conditioning and operational alignment. Conditioning on a single LLM draw per intent (`shared_initial`) corresponds to operational policies that constrain model calls for cost, latency, or governance reasons. Under such policies the guarded pipeline measures only deterministic validator and bounded repair rescue applied to the same candidate. Workflows that permit resampling, verifier-guided generation, or interactive steering change the estimand and typically increase achievable success rates by exploiting generator variance; those workflows must be evaluated under a different preregistered estimand.

Ceiling effects and the omitted pilot. The preregistration targeted detection of a 10 percentage-point absolute improvement with planned power ≈ 0.8 based on a pilot to estimate discordance. Because the optional pilot ($n = 60$) intended to estimate discordance was not executed, the main run confronted a near-ceiling baseline (299/300), producing minimal discordance ($m = 1$) and reduced power to detect small absolute differences. This practical outcome highlights the utility of small calibration pilots when generator performance may approach ceiling on chosen task suites.

Validator coverage and failure-mode dependence. The observed single failure mode was a sequencing omission; other classes (reachability violations, forbidden ACL removal) were not observed at scale on this synthetic suite. The marginal utility of a guarded pipeline therefore depends on (a) the deployment distribution of failure modes and (b) validator expressiveness: richer validators that detect reachability or ACL semantics may increase observed rescue rates. Conversely, if operator risk policies disallow automated edits, the guarded pipeline’s deterministic rescue becomes operationally limited to detection and human escalation.

Repair budget and bounded edits. We intentionally limited automated repair to a single constrained edit to measure bounded deterministic rescue. That design measured only localized sequencing corrections; iterative repair, larger edit operators, or interactive verifier-guided generation would increase rescue opportunities but address a different estimand. The archived single repair trace enables forensic analysis of edit efficacy and could inform permitted edit-operator design in production pipelines.

Practical implications for NetOps CI/CD. Organizations must align evaluation estimands with operational choices: if pipelines permit multiple model calls or verifier-guided regeneration, evaluations should allow resampling and interactive steering to measure the achievable success rates under those policies. If model calls are tightly budgeted, the `shared_initial` estimand provides an operationally meaningful upper bound for deterministic verifier/repair rescue applied to

a single candidate. Our artifact audit (`model_calls_used = 301`, `repair_stage_count = 1`) shows that in this run bounded repair produced measurable rescue at very low additional model cost for the single rescued episode.

VI. LIMITATIONS

- Synthetic benchmark: tasks and topologies were generated by the preregistered deterministic task generator; real-world heterogeneity may expose different failure modes and increase verifier marginal utility.
- Single model and validator: only `gpt-5-mini` and `deterministic_netops_validator_v5` were evaluated; results may not generalize to other LLMs or richer/diverse validators.
- Ceiling effect: baseline single-shot performance was near 100% on this suite (299/300), producing limited discordance and reduced power to detect small absolute improvements relative to larger pre-specified targets.
- Pilot not executed: the preregistered pilot intended for discordance estimation and power calibration was not run; no post-preregistration power recalculation was performed.
- Estimand conditioning: the `shared_initial` protocol conditions the estimand on a single LLM candidate and therefore does not measure verifier benefit that would arise from allowing independent alternative LLM candidates per condition (resampling or iterative generation).
- Repair budget: the guarded condition permitted at most one automated repair per pair; larger or iterative repair strategies may yield different outcomes.

VII. CONCLUSION

We executed a preregistered, artifact-archived paired comparison between a verifier-assisted pipeline (deterministic validator + ≤ 1 automated repair) and accepting a single-shot LLM candidate under the `shared_initial` estimand on 300 synthetic NetOps intents using `gpt-5-mini` and `deterministic_netops_validator_v5`. Baseline single-shot generation produced 299/300 unflagged verifier passes and the guarded pipeline 300/300, yielding an absolute paired improvement of 0.00333 (exact McNemar two-sided $p = 1.0$; 95% bootstrap CI [0.00,0.01]). Deterministic validator+repair rescued a single sequencing omission under the bounded repair budget. The near-ceiling baseline limits inferential sensitivity to small absolute improvements under the `shared_initial` estimand. We release full preregistration, execution, and analysis artifacts to support reproducibility and targeted follow-up studies that vary estimand conditioning, model strength, task distribution, or repair budget.

DISCLOSURE STATEMENT

This experiment and manuscript were produced by an autonomous CNSM Agentic-AI research run executed under the archived initial master prompt (provenance/master_prompt.txt, SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b39b15ba). Human role: the human Research Director

selected the topic family (Generative AI and LLMs for NetOps) and approved pre-lock infrastructure; all literature search after the autonomy lock, autonomous study selection, preregistration (study id `cnsmp2026-llm-verifier-predeployment-001`), execution, analysis, manuscript drafting, AI peer review, and final compilation were performed autonomously under the locked master prompt. Models and orchestration: the declared generation model used was `gpt-5-mini` (execution/model_configuration.json declares `requested_model = "gpt-5-mini"`); adapter/orchestration framework: `hosted_netops_gvr_v1`. Validator and tooling: `deterministic_netops_validator_v5` performed semantic and syntactic validation according to the preregistered `full_intent_constraint_validation` rule; the guarded condition permitted at most one automated repair call per pair implemented as a constrained edit operator. Preregistration: the confirmatory design, estimand, analysis executor, exclusion/missingness rules, and multiplicity plan were frozen prior to the main run and archived with the study id (preregistration SHA-256 = `425cb1652354120cf48f686b964723d64c76d0a8abe53cd5b5ad0eaa1af71fc7`). Execution summary (archived): `planned_episode_count = 600`, `completed_episode_count = 600`, `complete_pair_count = 300`, `model_calls_used = 301`; provider audit: `hosted_model_call_event_count = 301`, stage_counts show `shared_initial_generation = 300` and `repair = 1`. Pilot: the preregistration described an optional pilot ($n = 60$) for discordance estimation and power calibration; that pilot was not executed and no post-preregistration recalculation of required sample size was performed. Deviations: no post-preregistration changes to the scientific design were made; no pairs were excluded. Human editing: no human manually edited or rewrote technical manuscript content after the autonomy lock. Artifact availability: all raw outputs, logs, task manifests, validator traces, repair provider traces, and analysis artifacts referenced in this manuscript are archived in the run bundle associated with the study id. The archived run bundle contains the low-level repair provider trace documenting the single automated repair and subsequent validation pass; this manuscript reports a concise semantic summary of that repair in Results rather than reproducing full verbatim provider transcripts in the body.

REFERENCES

- [1] Yajie Zhou, Kevin Hsieh, Sathya Kumaran Mani, Srikanth Kandula, Zaoxing Liu, "MeshAgent: Enabling Reliable Network Management with Large Language Models", 2025, 10.1145/3771567
- [2] Xiaolong Wei, Yuehu Dong, Xingliang Wang, Xingyu Zhang, Zhejun Zhao, Dongdong Shen, Long Xia, Dawei Yin, "Beyond ReAct: A Planner-Centric Framework for Complex Tool-Augmented LLM Reasoning", 2026, 10.1609/aaai.v40i40.40676
- [3] Xu Liu, Peng Zhang, Anubhavnidhi Abhashkumar, Jiawei Chen, Weirong Jiang, "Automatic Configuration Repair", 2024, 10.1145/3696348.3696895
- [4] Hao Zhou, Chengming Hu, Ye Yuan, Yufei Cui, Yili Jin, Can Chen, Haolun Wu, Dun Yuan, Li Jiang, Di Wu, Xue Liu, Jianzhong Zhang, Xianbin Wang, Jiangchuan Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities", 2024, 10.1109/comst.2024.3465447
- [5] X X Zhang, Xianming Gao, Peilin Tao, Tao Feng, "GraphSynth: Synthesis of Network Configuration Templates Using Large Language Models", 2025, 10.1145/3728725.3728742
- [6] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, Yarin Gal, "Detecting hallucinations in large language models using semantic entropy", 2024, 10.1038/s41586-024-07421-0
- [7] Yunze Wei, Wei, Kaiwen, Shaohui Du, Wang, Jianyu, Liu, Zhangzhong, Yawen Wang, Zhenxin Li, Congcong Miao, Xiaohui Xie, Yong Cui, "Automated Network Protocol Testing with LLM Agents", 2025, 10.48550/arxiv.2510.13248
- [8] Fuliang Li, Haozhi Lang, Jiajie Zhang, Jiaxing Shen, Xingwei Wang, "PreConfig: A Pretrained Model for Automating Network Configuration", 2024, 10.48550/arxiv.2403.09369